

I am working on my Final Year Project in Electronics and Instrumentation Engineering.

PROJECT TITLE:

Smart Rice Disease Detection and Pesticide Recommendation System

DOMAIN:

Deep Learning, Computer Vision, Machine Learning, and Smart Agriculture.

CURRENT PROJECT SCOPE:

For now, I only want to develop the COMPLETE SOFTWARE/ML MODEL on my laptop.

Do NOT include hardware implementation, microcontrollers, pumps, valves, sensors, sprinkler control, or physical instrumentation at this stage.

Hardware/instrumentation may be integrated later after the software system is complete.

FINAL SOFTWARE PIPELINE:

Rice Leaf Image

- Data Preprocessing
- Disease Classification
- Diseased Area Segmentation
- Affected Area / Disease Severity Calculation
- Verified Disease-Treatment Knowledge Base
- Pesticide Recommendation
- Spray Timing / Application Recommendation
- Pesticide Quantity Calculation
- Final Laptop-Based Output

IMPORTANT PROJECT CHANGE:

Do NOT include plant growth-stage prediction as an ML task.

I currently have images of grown rice plants/leaves and do not have a labelled dataset containing different rice growth stages.

If the growth stage or plant age affects whether a pesticide should be sprayed, use scientifically verified agricultural recommendations/rules instead of training a growth-stage prediction model.

Similarly, pesticide recommendation should NOT be treated as a separate ML prediction problem unless there is a justified reason.

The disease-classification model should identify the disease.

Then a verified agricultural knowledge base should map:

Detected Disease

→ Recommended Treatment/Pesticide

→ Recommended Dose

→ Suitable Application Timing

→ Important Restrictions/Conditions

This information should be collected from reliable agricultural sources such as ICAR, government agricultural universities, official pesticide label recommendations, and other authoritative agricultural sources.

Do not invent pesticide recommendations or dosage information.

CURRENT STATUS:

ONLY DATA CLEANING HAS BEEN COMPLETED.

Do not assume preprocessing, dataset splitting, augmentation, model training, benchmarking, segmentation, treatment recommendation, or quantity calculation has been completed.

DATASET:

The cleaned rice leaf dataset contains 1,941 unique images belonging to six classes:

1. bacterial_leaf_blight
2. brown_spot
3. healthy
4. leaf_blast
5. leaf_scald
6. narrow_brown_spot

Most original images are approximately 1600 x 1600 pixels.

DATA CLEANING ALREADY COMPLETED:

- Dataset was inspected.
- Duplicate images were identified.
- Duplicate images were handled.
- Final clean dataset contains 1,941 unique images.
- Duplicate removal resulted in slightly unequal class sizes.

Clean dataset path:

/content/drive/MyDrive/RiceLeafsDisease/clean_dataset

An earlier 1,552 train / 389 test split was created, but do not automatically treat this as the final experimental split.

Determine the appropriate final splitting strategy before training.

=====

PHASE 1 - DATASET PREPARATION

=====

STEP 1 - VERIFY CLEAN DATASET

Check:

- Total images
- Number of classes
- Images per class
- Class imbalance
- Image dimensions
- Corrupted/unreadable images

Do not repeat duplicate cleaning unless necessary.

STEP 2 - DATASET SPLITTING

Determine an appropriate:

Train / Validation / Test

strategy.

Explain why validation data is required for model development and why the test set should remain untouched until final evaluation.

Use stratified splitting where appropriate.

STEP 3 - IMAGE PREPROCESSING

Determine appropriate preprocessing for pretrained CNN models.

Consider:

- Image resizing
- RGB conversion
- Tensor conversion
- Normalization
- Required input resolution

The original images are approximately 1600 x 1600.

Do not train directly at 1600 x 1600 unless there is a justified reason.

STEP 4 - DATA AUGMENTATION

Determine safe augmentation for rice disease images.

Consider:

- Rotation
- Horizontal flipping
- Vertical flipping if appropriate
- Cropping
- Brightness/color variation

Use augmentation only for training images.

Do not use transformations likely to remove or distort disease symptoms.

STEP 5 - PYTORCH DATALOADERS

Create training, validation, and test DataLoaders.

Choose an appropriate batch size based on available GPU memory.

=====

PHASE 2 - DISEASE CLASSIFICATION

=====

Use PyTorch and transfer learning.

Do not benchmark an unnecessarily large number of models.

Initially benchmark these representative pretrained CNNs:

1. MobileNetV3-Small
2. EfficientNet-B0
3. ResNet18
4. ResNet50

These represent different model capacities.

The purpose is to determine whether additional CNN capacity meaningfully improves performance on approximately 1,900 images.

STEP 6 - PREPARE PRETRAINED MODELS

For every architecture:

- Load pretrained ImageNet weights.
- Replace the classification layer for 6 classes.
- Understand total parameters.
- Understand trainable parameters.
- Decide which layers should initially be frozen.

STEP 7 - FIX INITIAL TRAINING CONFIGURATION

Keep training conditions reasonably consistent across architectures.

Determine:

- Loss function
- Optimizer
- Learning rate

- Batch size
- Epochs
- Scheduler if required
- Early stopping if required

CrossEntropyLoss and Adam/AdamW may be used as reasonable initial choices.

Do not perform extensive optimizer/hyperparameter tuning yet.

Architecture benchmarking comes first.

STEP 8 - TRAIN MODELS

Train each architecture under comparable conditions.

Record:

- Training loss
- Validation loss
- Training accuracy
- Validation accuracy
- Training time

Monitor overfitting and underfitting.

STEP 9 - MODEL BENCHMARKING

Create a comparison containing:

- Model
- Total parameters
- Trainable parameters
- Training accuracy
- Validation accuracy
- Macro precision
- Macro recall
- Macro F1
- Training time
- Model size
- Inference time if useful
- Overfitting observations

Main experimental question:

Does increasing pretrained CNN capacity improve rice disease classification on this dataset, and at what point does additional model capacity stop providing meaningful improvement?

STEP 10 - MODEL EVALUATION

For promising models calculate:

- Test accuracy
- Precision
- Recall
- F1 score
- Confusion matrix
- Per-class performance

Also inspect misclassified images to understand which diseases are being confused.

STEP 11 - MODEL SELECTION

Select the disease classifier using:

- Generalization performance
- F1 score
- Per-class performance
- Overfitting
- Parameter count
- Model size
- Inference speed

Do not select the final model based only on accuracy.

STEP 12 - HYPERPARAMETER TUNING

After selecting one or two promising architectures, investigate:

- Learning rate
- Optimizer
- Weight decay
- Batch size
- Scheduler
- Augmentation
- Fine-tuning depth

Do not extensively tune architectures that already performed poorly.

STEP 13 - FINAL DISEASE CLASSIFIER

Train the selected configuration and save:

- Model weights
- Class names
- Preprocessing configuration
- Training configuration
- Final evaluation metrics

=====

PHASE 3 - DISEASED AREA SEGMENTATION

=====

Classification only tells us which disease is present.

The next objective is to determine how much of the leaf is affected.

The desired output is a disease mask identifying diseased and healthy portions of the leaf.

Investigate suitable segmentation approaches such as:

- U-Net
- DeepLab
- Other appropriate segmentation architectures

Before training segmentation, determine whether the current dataset contains segmentation masks.

If masks are unavailable, determine a realistic method for obtaining or creating labelled masks.

Do not pretend that classification labels provide pixel-level affected-area information.

STEP 14 - SEGMENTATION MODEL

Develop and evaluate the segmentation system.

Possible metrics:

- IoU
- Dice score
- Pixel accuracy

STEP 15 - AFFECTED AREA CALCULATION

Using the segmentation output, calculate:

Affected Area Percentage =
 $\text{Diseased Leaf Pixels} / \text{Total Leaf Pixels} \times 100$

Determine whether leaf-background segmentation is also necessary to avoid counting image background.

Use affected percentage to represent disease severity where scientifically appropriate.

=====

PHASE 4 - AGRICULTURAL KNOWLEDGE BASE

=====

This should NOT be another neural network unless necessary.

Research reliable agricultural sources for each disease:

- bacterial_leaf_blight
- brown_spot
- leaf_blast
- leaf_scald
- narrow_brown_spot

For every disease collect verified information about:

- Recommended management
- Recommended pesticide/fungicide where applicable
- Active ingredient
- Recommended concentration/dose
- Application timing
- Number/frequency of applications if officially specified
- Relevant crop-stage restrictions
- Safety or usage restrictions
- Source/reference

Prefer authoritative sources such as:

- ICAR
- ICAR-NCIPM
- Government agriculture departments

- Agricultural universities
- Official registered pesticide label information

Do not invent missing values.

If recommendations differ by region or source, record the differences rather than silently choosing one.

Also consider Integrated Pest Management.

Chemical treatment should not automatically be recommended simply because a disease is detected.

=====

PHASE 5 - TREATMENT DECISION

=====

Combine:

Disease Classification

+

Affected Area / Severity

+

Verified Agricultural Knowledge

to generate a treatment recommendation.

Example conceptual logic:

Disease detected

- Determine severity
- Look up recommended management
- Determine whether spraying is appropriate
- Select verified treatment
- Retrieve recommended dosage/application information

The exact rules must be based on agricultural evidence.

=====

PHASE 6 - PESTICIDE QUANTITY CALCULATION

=====

After obtaining verified dosage information, calculate the required pesticide quantity.

Potential inputs may include:

- Recommended dose/concentration
- Spray volume
- Area to be treated
- Disease severity where applicable
- Manufacturer/official application instructions

Clearly separate:

ML prediction

from

deterministic pesticide quantity calculation.

The CNN should not invent pesticide quantities.

=====

PHASE 7 - LAPTOP APPLICATION

=====

After all components work independently, combine them into one laptop-based application.

Expected user workflow:

Upload Rice Leaf Image

- Display Predicted Disease
- Display Confidence
- Display Segmentation Mask
- Display Affected Area Percentage
- Display Disease Severity
- Display Recommended Management
- Display Pesticide/Fungicide if appropriate
- Display Recommended Application Information
- Display Calculated Quantity if enough inputs are available
- Display Agricultural Source/Reference

The application can initially run entirely on my laptop.

No hardware implementation is required at this stage.

=====

WORKING STYLE

=====

Guide me STEP BY STEP.

Do not provide the entire project code at once.

For every step:

- 1. Explain what we are doing.
- 2. Explain why it is required.
- 3. Explain important choices.
- 4. Give only the code required for that step.
- 5. Let me execute it.
- 6. I will provide the output.
- 7. Analyze the output.
- 8. Decide the next step based on the result.

Do not silently make major project decisions.

If several approaches are reasonable, explain the important differences and let me decide.

Do not repeatedly revisit completed work unless a problem is found.

=====

PROJECT DOCUMENTATION

=====

Keep track of:

- Dataset problems
- Duplicate handling
- Class distribution
- Dataset splitting
- Preprocessing
- Augmentation
- Model architectures
- Parameter counts
- Training configuration
- Benchmark results
- Overfitting/underfitting
- Hyperparameter changes
- Final model-selection reasoning
- Segmentation methodology
- Segmentation results
- Affected-area calculation
- Agricultural sources
- Disease-treatment mappings
- Dosage sources
- Treatment-decision logic
- Quantity-calculation methodology
- Final application design
- Limitations
- Future hardware integration

These should later be compiled into my final-year project report.

MOST IMPORTANT CURRENT INSTRUCTION:

ONLY DATA CLEANING HAS BEEN COMPLETED.

Start from the immediate next step after data cleaning.

Do not assume that preprocessing, dataset splitting, training, benchmarking, segmentation, pesticide recommendation, or application development has already been completed.